Deep Standard

VOL. 1, NO. 016

2026-03-30

COVER STORY

Fragments: March 10

If I was giving the keynote at SRECon 2026, I would ditch the begrudging stance.

Martin Fowler

Tech firm fined $1.1m by California for selling high-school students' data

I agree with Brian Marick's response

No such story should be published without a comparison of the fine to the company's previous year revenue and profits, or valuation of last funding round. (I could only find a valuation of $11.0M in 2017.)

We desperately need corporations' attitudes to shift from "lawbreaking is a low-risk cost of doing business; we get a net profit anyway" to "this could be a death sentence."

❄ ❄ ❄ ❄ ❄

Charity Majors gave the closing keynote at SRECon last year, encouraging people to engage with generative AI.

If I was giving the keynote at SRECon 2026, I would ditch the begrudging stance. I would start by acknowledging that AI is radically changing the way we build software. It's here, it's happening, and it is coming for us all.

Her agenda this year would be to tell everyone that they mustn't wait for the wave to crash on them, but to swim out to meet it. In particular, I appreciated her call to resist our confirmation bias:

The best advice I can give anyone is: know your nature, and lean against it.

If you are a reflexive naysayer or a pessimist, know that, and force yourself to find a way in to wonder, surprise and delight.

If you are an optimist who gets very excited and tends to assume that everything will improve: know that, and force yourself to mind real cautionary tales.

❄ ❄ ❄ ❄ ❄

In a comment to Kief Morris's recent article on Humans and Agents in Software Loops , in LinkedIn comments Renaud Wilsius may have coined another bit of terminology for the agent+programmer age

This completes the story of productivity, but it opens a new chapter on talent: The Apprentice Gap. If we move humans 'on the loop' too early in their careers, we risk a future where no one understands the 'How' deeply enough to build a robust harness. To manage the flywheel effectively, you still need the intuition that comes from having once been 'in the loop.' The next great challenge for CTOs isn't just Harness Engineering, it's 'Experience Engineering' for our junior developers in an agentic world.

❄ ❄ ❄ ❄ ❄

In hearing conversations about "the ralph loop", I often hear it in the sense of just letting the agents loose to run on their own. So it's interesting to read the originator of the ralph loop point out:

It's important to watch the loop as that is where your personal development and learning will come from. When you see a failure domain – put on your engineering hat and resolve the problem so it never happens again.

In practice this means doing the loop manually via prompting or via automation with a pause that involves having to prcss CTRL+C to progress onto the next task. This is still ralphing as ralph is about getting the most out how the underlying models work through context engineering and that pattern is GENERIC and can be used for ALL TASKS.

At the Thoughtworks Future of Software Development Retreat we were very concerned about cognitive debt. Watching the loop during ralphing is a way to learn about what the agent is building, so that it can be directed effectively in the future.

❄ ❄ ❄ ❄ ❄

Anthropic recently published a page on how AI helps break the cost barrier to COBOL modernization . Using AI to help migrate COBOL systems isn't an new idea to my colleagues, who shared their experiences using AI for this task over a year ago. While Anthropic's article is correct about the value of AI, there's more to the process than throwing some COBOL at an LLM.

The assumption that AI can simply translate COBOL into Java treats modernization as a syntactic exercise, as though a system is nothing more than its source code. That premise is flawed.

A direct translation would, in the best case scenario, faithfully reproduce existing architectural constraints, accumulated technical debt and outdated design decisions. It wouldn't address weaknesses; it would restate them in a different language.

...

In practice, modernization is rarely about preserving the past in a new syntax. It's about aligning systems with current market demands, infrastructure paradigms, software supply chains and operating models. Even if AI were eventually capable of highly reliable code translation, blind conversion would risk recreating the same system with the same limitations, in another language, without a deliberate strategy for replacing or retiring its legacy ecosystem.

❄ ❄ ❄ ❄ ❄

Anders Hoff (inconvergent)

an LLM is a compiler in the same way that a slot machine is an ATM

❄ ❄ ❄ ❄ ❄

One of the more interesting aspects of the network of people around Jeffrey Epstein is how many people from academia were connected. It's understandable why, he had a lot of money to offer, and most academics are always looking for funding for their work. Most of the attention on Epstein's network focused on those that got involved with him, but I'm interested in those who kept their distance and why - so I enjoyed Jeffrey Mervis's article in Science

Many of the scientists Epstein courted were already well-established and well-funded. So why didn't they all just say no? Science talked with three who did just that. Here's how Epstein approached them, and why they refused to have anything to do with him.

I believe that keeping away from bad people makes life much more pleasant, if nothing else it reduces a lot of stress. So it's good to understand how people make decisions on who to avoid.

{

FEATURES

Analysing Optimisations in the Wire Serialiser

Matt Warren (.NET)

Recently Roger Johansson wrote a post titled Wire – Writing one of the fastest . NET serializers , describing the optimisation that were implemented to make Wire as fast as possible. He also followed up that post with a set of benchmarks , showing how Wire compared to other . NET serialisers:

Using BenchmarkDotNet , this post will analyse the individual optimisations and show how much faster each change is. For reference, the full list of optimisations in the original blog post are:

Looking up value serializers by type

Looking up types when deserializing

Byte buffers, allocations and GC

Clever allocations

Boxing, Unboxing and Virtual calls

Fast creation of empty objects

id="looking-up-value-serializers-by-type">Looking up value serializers by type

This optimisation changed code like this:

class="highlight"> public ValueSerializer GetSerializerByType (Type type) { ValueSerializer serializer ;

if (_serializers . TryGetValue (type , out serializer)) return serializer ;

//more code to build custom type serializers.. ignore for now. }

into this:

class="highlight"> public ValueSerializer GetSerializerByType (Type type) { if (ReferenceEquals (type . GetTypeInfo (). Assembly , ReflectionEx . CoreAssembly)) { if (type == TypeEx . StringType) //we simply keep a reference to each primitive type return StringSerializer . Instance ;

if (type == TypeEx . Int32Type) return Int32Serializer . Instance ;

if (type == TypeEx . Int64Type) return Int64Serializer . Instance ; ... }

So it has replaced a dictionary lookup with an if statement. In addition it is caching the Type instance of known types, rather than calculating them every time. As you can see the optimisation pays off in some circumstance but not in others, so it's not a clear win. It depends on where the type is in the list of if statements. If it's near the beginning (e.g. System. String) it'll be quicker than if it's near the end (e.g. System. Byte[]), which makes sense as all the other comparisons have to be done first.

Full benchmark code and results

id="looking-up-types-when-deserializing">Looking up types when deserializing

The 2nd optimisation works by removing all unnecessary memory allocations, it did this by:

Using a custom struct (value type) rather than a class

Pre-calculating a hash code once, rather than each time a comparison is needed.

Doing string comparisons with raw byte [] , rather than deserialising to a string

Full benchmark code and results

Note: these results nicely demonstrate how BenchmarkDotNet can show you memory allocations as well as the time taken.

Interestingly they hadn't actually removed all memory allocations as the comparisons between OptimisedLookup and OptimisedLookupCustomComparer show. To fix this I sent a P. R which removes unnecessary boxing, by using a Custom Comparer rather than the default struct comparer.

id="byte-buffers-allocations-and-gc">Byte buffers, allocations and GC

Again removing unnecessary memory allocations were key in this optimisation, most of which can be seen in the NoAllocBitConverter . Clearly serialisation spends a lot of time converting from the in-memory representation of an object to the serialised version, i.e. a byte [] . So several tricks were employed to ensure that temporary memory allocations were either removed completely or if that wasn't possible, they were done by re-using a buffer from a pool rather than allocating a new one each time (see "Buffer recycling")

Full benchmark code and results

id="clever-allocations">Clever allocations

This optimisation is perhaps the most interesting, because it's implemented by creating a custom data structure, tailored to the specific needs of Wire. So, rather than using the default . NET dictionary , they implemented FastTypeUShortDictionary . In essence this data structure optimises for having only 1 item, but falls back to a regular dictionary when it grows larger. To see this in action, here is the code from the TryGetValue(..) method :

class="highlight"> public bool TryGetValue (Type key , out ushort value) { switch (_length) { case 0 : value = 0 ; return false ; case 1 : if (key == _firstType) { value = _firstValue ; return true ; } value = 0 ; return false ; default : return _all . TryGetValue (key , out value); } }

```
}
{
```

Why is reflection slow?

Matt Warren (.NET)

It's common knowledge that reflection in . NET is slow , but why is that the case? This post aims to figure that out by looking at what reflection does under-the-hood .

id="clr-type-system-design-goals">CLR Type System Design Goals

But first it's worth pointing out that part of the reason reflection isn't fast is that it was never designed to have high-performance as one of its goals, from Type System Overview - 'Design Goals and Non-goals' :

Accessing information needed at runtime from executing (non-reflection) code is very fast.

Accessing information needed at compilation time for generating code is straightforward.

The garbage collector/stackwalker is able to access necessary information without taking locks, or allocating memory.

Minimal amounts of types are loaded at a time.

Minimal amounts of a given type are loaded at type load time.

Type system data structures must be storable in NGEN images.

Non-Goals

All information in the metadata is directly reflected in the CLR data structures.

All uses of reflection are fast.

and along the same lines, from Type Loader Design - 'Key Data Structures' :

EEClass

MethodTable data are split into "hot" and "cold" structures to improve working set and cache utilization. MethodTable itself is meant to only store "hot" data that are needed in program steady state. EEClass stores "cold" data that are typically only needed by type loading, JITing or reflection. Each MethodTable points to one EEClass.

id="how-does-reflection-work">How does Reflection work?

So we know that ensuring reflection was fast was not a design goal, but what is it doing that takes the extra time?

Well there several things that are happening, to illustrate this lets look at the managed and unmanaged code callstack that a reflection call goes through.

System. Reflection. RuntimeMethodInfo. Invoke (..) - source code link

calling System. Reflection. RuntimeMethodInfo. UnsafeInvokeInternal (..)

System. RuntimeMethodHandle. PerformSecurityCheck (..) - link

calling System. GC. KeepAlive (..)

System. Reflection. RuntimeMethodInfo. UnsafeInvokeInternal (..) - link

calling stub for System. RuntimeMethodHandle. InvokeMethod (..)

stub for System. RuntimeMethodHandle. InvokeMethod (..) - link

Even if you don't click the links and look at the individual C#/cpp methods, you can intuitively tell that there's alot of code being executed along the way. But to give you an example, the final method, where the bulk of the work is done, System. RuntimeMethodHandle. InvokeMethod is over 400 LOC !

But this is a nice overview, however what is it specifically doing?

id="fetching-the-method-information">Fetching the Method information

Before you can invoke a field/property/method via reflection you have to get the FieldInfo/PropertyInfo/MethodInfo handle for it, using code like this:

As shown in the previous section there's a cost to this, because the relevant meta-data has to be fetched, parsed, etc. Interestingly enough the runtime helps us by keeping an internal cache of all the fields/properties/methods. This cache is implemented by the RuntimeTypeCache class and one example of its usage is in the RuntimeMethodInfo class .

You can see the cache in action by running the code in this gist , which appropriately enough uses reflection to inspect the runtime internals!

Before you have done any reflection to obtain a FieldInfo , the code in the gist will print this:

But once you've fetched even just one field, then the following will be printed:

class="highlight"> Type: ReflectionOverhead. Program Reflection Type: System. RuntimeType (BaseType: System. Reflection. TypeInfo) RuntimeTypeCache: System. RuntimeType+RuntimeTypeCache, m_cacheComplete = True, 4 items in cache [0] - Int32 TestField1 - Private [1] - System. String TestField2 - Private [2] - Int32 k__BackingField - Private [3] - System. String TestField3 - Private, Static

```
}
{
```

Know thy .NET object memory layout (Updated 2014-09-03)

Matt Warren (.NET)

Apologies to Nitsan Wakart , from whom I shamelessly stole the title of this post !

tl;dr

The . NET port of HdrHistogram can control the field layout within a class, using the same technique that the original Java code does.

Recently I've spent some time porting HdrHistogram from Java to . NET , it's been great to learn a bit more about Java and get a better understanding of some low-level code. In case you're not familiar with it, the goals of HdrHistogram are to:

Provide an accurate mechanism for measuring latency at a full-range of percentiles (99.9%, 99.99% etc)

Minimising the overhead needed to perform the measurements, so as to not impact your application

You can find a full explanation of what is does and how point 1) is achieved in the project readme .

Minimising overhead

But it's the 2nd of the points that I'm looking at in this post, by answering the question

How does HdrHistogram minimise its overhead?

But first it makes sense to start with the why, well it turns out it's pretty simple. HdrHistogram is meant for measuring low-latency applications, if it had a large overhead or caused the GC to do extra work, then it would negatively affect the performance of the application is was meant to be measuring.

Also imagine for a minute that HdrHistogram took 1/10,000th of a second (0.1 milliseconds or 100,000 nanoseconds) to record a value. If this was the case you could only hope to accurately record events lasting down to a millisecond (1/1,000th of a second), anything faster would not be possible as the overhead of recording the measurement would take up too much time.

As it is HdrHistogram is much faster than that, so we don't have to worry! From the readme :

Measurements show value recording times as low as 3-6 nanoseconds on modern (circa 2012) Intel CPUs.

So how does it achieve this, well it does a few things:

It doesn't do any memory allocations when storing a value, all allocations are done up front when you create the histogram. Upon creation you have to specify the range of measurements you would like to record and the precision. For instance if you want to record timings covering the range from 1 nanosecond (ns) to 1 hour (3,600,000,000,000 ns), with 3 decimal places of resolution, you would do the following:

Histogram histogram = new Histogram(3600000000000L, 3);

Uses a few low-level tricks to ensure that storing a value can be done as fast as possible. For instance putting the value in the right bucket (array location) is a constant lookup (no searching required) and on top of that it makes use of some nifty bit-shifting to ensure it happens as fast as possible.

Implements a slightly strange class-hierarchy to ensure that fields are laid out in the right location. It you look at the source you have AbstractHistogram and then the seemingly redundant class AbstractHistogramBase , why split up the fields up like that? Well the comments give it away a little bit, it's due to false-sharing

False sharing

Update (2014-09-03): As pointed out by Nitsan in the comments , I got the wrong end of the stick with this entire section. It's not about false-sharing at all, it's the opposite, I'll quote him to make sure I get it right this time!

The effort made in HdrHistogram towards controlling field ordering is not about False Sharing but rather towards ensuring certain fields are more likely to be loaded together as they are clumped together, thus avoiding a potential extra read miss.

So what is false sharing, to find out more I recommend reading Martin Thompson's excellent post and this equally good one from Nitsan Wakart. But if you're too lazy to do that, it's summed up by the image below (from Martin's post).

The opposite is also true, you can gain performance by ensuring that fields you know are accessed in succession are located together in memory.

Matt Warren (.NET)

The problem is that a CPU pulls data into its cache in lines, even if your code only wants to read a single variable/field. If 2 threads are reading from 2 fields (X and Y in the image) that are next to each other in memory, the CPU running a thread will invalidate the cache of the other CPU when it pulls in a line of memory. This invalidation costs time and in high-performance situations can slow down your program.

The opposite is also true, you can gain performance by ensuring that fields you know are accessed in succession are located together in memory. This means that once the first field is pulled into the CPU cache, subsequent accesses will be cheaper as the fields will be "Hot" . It is this scenario HdrHistogram is trying to achieve, but how do you know that fields in a . NET object are located together in memory?

```
}
{
```

Research based on the .NET Runtime

Matt Warren (.NET)

Over the last few years, I've come across more and more research papers based, in some way, on the 'Common Language Runtime' (CLR).

Note: I put the papers into the following categories to make them easier to navigate (papers in each category are sorted by date, newest -> oldest):

Using the . NET Runtime as a case-study

to prove its correctness , study how it works or analyse its behaviour

" It was formed in 1991, with the intent to advance state-of-the-art computing and solve difficult world problems through technological innovation in collaboration with academic, government, and industry researchers " (according to Wikipedia)

Papers based on the Mono Runtime

a ' Cross-Platform, open-source . NET framework '

Using 'Rotor' , real name 'Shared Source CLI (SSCLI)'

from Wikipedia " Microsoft provides the Shared Source CLI as a reference CLI implementation suitable for educational use "

Any papers I've missed? If so, please let me know in the comments or on Twitter

. NET Runtime as a Case-Study

Pitfalls of C# Generics and Their Solution Using Concepts (Belyakova & Mikhalkovich, 2015)

Efficient Compilation of . NET Programs for Embedded Systems (Sallenaveab & Ducournaub, 2011)

Type safety of C# and . Net CLR (Fruja, 2007)

Modeling the . NET CLR Exception Handling Mechanism for a Mathematical Analysis (Fruja & Börger, 2006)

Analysis of the . NET CLR Exception Handling Mechanism (Fruja & Börger, 2005)

A Modular Design for the Common Language Runtime (CLR) Architecture (Fruja, 2005)

Cross-language Program Slicing in the . NET Framework (Pócza, Biczó & Porkoláb, 2005)

Design and Implementation of a high-level multi-language . NET Debugger (Strein, 2005)

A High-Level Modular Definition of the Semantics of C# (Börger, Fruja, Gervasi & Stärk, 2004)

An ASM Specification of C# Threads and the . NET Memory Model (Stärk and Börger, 2004)

Common Language Runtime : a new virtual machine (Ferreira, 2004)

JVM versus CLR: a comparative study (Singer, 2003)

Runtime Code Generation with JVM And CLR (Sestoft, 2002)

Project Snowflake: Non-blocking safe manual memory management in . NET (Parkinson, Vaswani, Costa, Deligiannis, Blankstein, McDermott, Balkind & Vytiniotis, 2017)

Simple, Fast and Safe Manual Memory Management (Kedia, Costa, Vytiniotis, Parkinson, Vaswani & Blankstein, 2017)

Uniqueness and Reference Immutability for Safe Parallelism (Gordon, Parkinson, Parsons, Bromfield & Duffy, 2012)

A study of concurrent real-time garbage collectors (Pizlo, Petrank & Steensgaard, 2008)

Optimizing concurrency levels in the. net threadpool: A case study of controller design and implementation (Hellerstein, Morrison & Eilebrecht, 2008)

Stopless: a real-time garbage collector for multiprocessors. (Pizlo, Frampton, Petrank & Steensgaard, 2007)

Securing the . NET Programming Model (Kennedy, 2006)

Combining Generics, Pre-compilation and Sharing Between Software-Based Processes (Syme & Kennedy, 2004)

Formalization of Generics for the . NET Common Language Runtime (Yu, Kennedy & Syme, 2004)

Runtime Verification of . NET Contracts (Barnett & Schulte, 2003)

Design and Implementation of Generics for the . NET Common Language Runtime (Kennedy & Syme, 2001)

Typing a Multi-Language Intermediate Code (Gordon & Syme, 2001)

Static and Dynamic Analysis of Android Malware and Goodware Written with Unity Framework (Shim, Lim, Cho, Han & Park, 2018)

Reducing startup time of a deterministic virtualizing runtime environment (Däumler & Werner, 2013)

Detecting Clones Across Microsoft . NET Programming Languages (Al-Omari, Keivanloo, Roy & Rilling, 2012)

Language-independent sandboxing of just-in-time compilation and self-modifying code (Ansel & Marchenko, 2012)

VMKit: a Substrate for Managed Runtime Environments (Geoffray, Thomas, Lawall, Muller & Folliot, 2010)

```
}
{
```

Felix, the “DevOps” between designers and developers

Junmin Liu · Groupon Engineering

— Building Felix, the Design System for Groupon

Several new features have been released on Groupon.com recently, such as the QR code in the navigation bar to download the app, and a banner carousel to display multiple banner messages within a single view.

In the past, similar product features might take 2–3 sprints to complete, but now all of these features are developed and tested in a single sprint. The reason why we can finish the development and testing efficiently is mainly credited to Felix, the design system we’re building for Groupon.

What is a design system?

Design Systems are not new to the world of product development. There are many successful cases in the industry, such as Google’s Material Design, Microsoft’s Fluent Design System, IBM’s Carbon, Adobe’s Spectrum, and so on. With their design systems, these companies have successfully changed the way they design and develop their software products by establishing a set of reusable components and usage guidelines to inform design and development. The result? Delivering faster and better product experiences to our customers, at scale.

Felix, the early days

In 2020, Groupon’s consumer experience design and engineering teams began partnering to bring to life a comprehensive design system–now named Felix–to be used throughout Groupon’s many brands, platforms, and products.

Initially, I believed a design system to be simply a set of design specifications and component libraries — an unoriginal concept. After diving into the architectural design and project management of Felix’s development with Lead Product Designer, Michelle Witkowski, I realized Felix’s true potential: a fundamental change in collaboration between designers and developers.

This year, we have built out the initial implementation of Felix, and several of our development teams have begun leveraging it. Through the use of design tokens within Felix, we have developed themes to support Groupon’s multiple brands (Groupon and LivingSocial) and platforms (web and mobile apps). These theme styles can be changed at scale, lending support to even major brand update initiatives. Having a design system in place greatly improves our design and development efficiency, enables product strategy acceleration, and ensures that what we build is consistent with our brand vision.

“The space to solve the tough problems really opens up when the rest is simply pointing to the right component.” — Michael Forsythe

Felix is helping to build a culture of collaboration between Designers, Developers, and Product

When we talk about DevOps, one of the most exciting concepts is the way developers and operations work together closely to ensure applications can be efficiently built, tested, and released.

Similarly, when we talk about Felix, we must talk about the collaboration between designers, developers, and product managers. Like DevOps, a design system uncovers a culture of collaboration between product delivery’s most prominent teams and introduces the kind of seamless product development cycle once only enjoyed by DevOps.

Designers and Developers can share a common language: Design Tokens

Before there was Felix, designers and developers were working in their own languages, in their own ecosystems, and it was difficult to communicate with each other.

The end result, most of the time, was that we ended up spending time doing pixel perfection activities after a particular UI design was implemented by a developer. This usually happened because the developers did not understand the design well, and it required both teams to communicate several times to achieve the same result as the initial design.

So, one of the most important things when building a design system is to define the design tokens. At Groupon, we share a set of design tokens for both development and design, regardless of the platform.

A Design Token is essentially a set of key value combinations. For example, the following figure is a list of color design tokens, simple and clear. Another way to think of them is they represent codified design decisions.

Design Tokens, Credit: Michelle Witkowski

With Design Tokens, it is simpler for designers to describe their designs and easier for developers to understand them.

“I think working together has allowed me to better understand the focus points of designers in a new feature, so the overall process of creating something new has less iterations — a more streamlined process.” — Nida Pervez

“Working together on the Design System has helped give designers and engineers an extra common language. This boost in communication helps to increase productivity across teams.” — Daniel Hey

Compare these two pictures below, the first one is before Design Tokens and the other one is after Design Tokens, the difference is very obvious — with the guesswork taken out of the process, developers can simply reference the tokens defined by the design team.

Building a Design System Team

```
}
{
```

Fragments: February 9

Martin Fowler

Some more thoughts from last week's open space gathering on the future of software development in the age of AI. I haven't attributed any comments since we were operating under the Chatham House Rule , but should the sources recognize themselves and would like to be attributed, then get in touch and I'll edit this post.

❄ ❄

During the opening of the gathering, I commented that I was naturally skeptical of the value of LLMs. After all, the decades have thrown up many tools that have claimed to totally change the nature of software development. Most of these have been little better than snake oil.

But I am a total, absolute skeptic - which means I also have to be skeptical of my own skepticism.

❄ ❄

One of our sessions focused on the problem of "cognitive debt". Usually, as we build a software system, the developers of that system gain an understanding both the underlying domain and the software they are building to support it. But once so much work is sent off to LLMs, does this mean the team no longer learns as much? And if so, what are the consequences of this? Can we rely on The Genie to keep track of everything, or should we take active measures to ensure the team understands more of what's being built and why?

The TDD cycle involves a key (and often under-used) step to refactor the code. This is where the developers consolidate their understanding and embed it into the codebase. Do we need some similar step to ensure we understand what the LLMs are up to?

When the LLM writes some complex code, ask it to explain how it works. Maybe get it do so in a funky way, such as asking it to explain the code's behavior in the form of a fairy tale.

❄ ❄

OH:

LLMs are drug dealers, they give us stuff, but don't care about the resulting system or the humans that develop and use it.

Who cares about the long-term health of the system when the LLM renews its context with every cycle?

❄ ❄

Programmers are wary of LLMs not just because folks are worried for their jobs, but also because we're scared that LLMs will remove much of the fun from programming. As I think about this, I consider what I enjoy about programming. One aspect is delivering useful features - which I only see improving as LLMs become more capable.

But, for me, programming is more than that. Another aspect I enjoy about programming is model building. I enjoy the process of coming up with abstractions that help me reason about the domain the code is supporting - and I am concerned that LLMs will cause me to spend less attention on this model building. It may be, however, that model-building becomes an important part of working effectively with LLMs, a topic Unmesh Joshi and I explored a couple of months ago.

❄ ❄

In the age of LLMs, will there still be such a things as "source code", and if so, what will it look like? Prompts, and other forms of natural language context can elicit a lot of behavior, and cause a rise in the level of abstraction, but also a sideways move into non-determinism . In all this is there still a role for a persistent statement of non-deterministic behavior?

Almost a couple of decades ago, I became interested in a class of tools called Language Workbenches . They didn't have a significant impact on software development, but maybe the rise of LLMs will reintroduce some ideas from them. These tools rely on a semantic model that the tool persists in some kind of storage medium, that isn't necessarily textual or comprehensible to humans directly. Instead, for humans to understand it, the tools include projectional editors that create human-readable projections of the model.

Could this notion of a non-human deterministic representation become the future source code? One that's designed to maximize expression with minimal tokens?

❄ ❄

OH:

Scala was the first example of a lab-leak in software. A language designed for dangerous experiments in type theory escaped into the general developer population.

❄ ❄ ❄ ❄ ❄

elsewhere on the web

Angie Jones on tips for open source maintainers to handle AI contributions

I've been seeing more and more open source maintainers throwing up their hands over AI generated pull requests. Going so far as to stop accepting PRs from external contributors.

[snip]

But yo, what are we doing?! Closing the door on contribu-

```
}
{
```

Is C# a low-level language?

Matt Warren (.NET)

I'm a massive fan of everything Fabien Sanglard does, I love his blog and I've read both his books cover-to-cover (for more info on his books, check out the recent Hansleminutes podcast).

Recently he wrote an excellent post where he deciphered a postcard sized raytracer , un-packing the obfuscated code and providing a fantastic explanation of the maths involved. I really recommend you take the time to read it!

But it got me thinking, would it be possible to port that C++ code to C#?

Partly because in my day job I've been having to write a fair amount of C++ recently and I've realised I'm a bit rusty, so I thought this might help!

But more significantly, I wanted to get a better insight into the question is C# a low-level language?

A slightly different, but related question is how suitable is C# for 'systems programming'? For more on that I really recommend Joe Duffy's excellent post from 2013 .

id="line-by-line-port">Line-by-line port

I started by simply porting the un-obfuscated C++ code line-by-line to C# . Turns out that this was pretty straight forward, I guess the story about C# being C++++ is true after all!!

Let's look at an example, the main data structure in the code is a 'vector', here's the code side-by-side, C++ on the left and C# on the right:

So there's a few syntax differences, but because . NET lets you define your own 'Value Types' I was able to get the same functionality. This is significant because treating the 'vector' as a struct means we can get better 'data locality' and the . NET Garbage Collector (GC) doesn't need to be involved as the data will go onto the stack (probably, yes I know it's an implementation detail).

For more info on structs or 'value types' in . NET see:

Heap vs stack, value type vs reference type

Value Types vs Reference Types

Memory in . NET - what goes where

The Truth About Value Types

The Stack Is An Implementation Detail, Part One

In particular that last post form Eric Lippert contains this helpful quote that makes it clear what 'value types' really are:

Surely the most relevant fact about value types is not the implementation detail of how they are allocated , but rather the by-design semantic meaning of "value type", namely that they are always copied "by value" . If the relevant thing was their allocation details then we'd have called them "heap types" and "stack types". But that's not relevant most of the time. Most of the time the relevant thing is their copying and identity semantics.

Now lets look at how some other methods look side-by-side (again C++ on the left, C# on the right), first up RayTracing(..) :

Next QueryDatabase(..) :

(see Fabien's post for an explanation of what these 2 functions are doing)

But the point is that again, C# lets us very easily write C++ code! In this case what helps us out the most is the ref keyword which lets us pass a value by reference . We've been able to use ref in method calls for quite a while, but recently there's been a effort to allow ref in more places:

Ref returns and ref locals

C# 7 Series, Part 9: ref structs

Now sometimes using ref can provide a performance boost because it means that the struct doesn't need to be copied, see the benchmarks in Adam Sitniks post and Performance traps of ref locals and ref returns in C# for more information.

However what's most important for this scenario is that it allows us to have the same behaviour in our C# port as the original C++ code. Although I want to point out that 'Managed References' as they're known aren't exactly the same as 'pointers', most notably you can't do arithmetic on them, for more on this see:

ref returns are not pointers

Managed pointers

References are not addresses

id="performance">Performance

So, it's all well and good being able to port the code, but ultimately the performance also matters. Especially in something like a 'ray tracer' that can take minutes to run! The C++ code contains a variable called sampleCount that controls the final quality of the image, with sampleCount = 2 it looks like this:

Which clearly isn't that realistic!

However once you get to sampleCount = 2048 things look a lot better:

But, running with sampleCount = 2048 means the rendering

}

standard-article(title: [Episode 919: The Who is the What; the When is the Why], author: [Matthew Wrather], source-name: [Overthinking It], images: (), paragraphs: ([style="text-align: center;"> Support Overthinking It by becoming a member for $5/month!], [Peter Fenzel, Mark Lee, and Matthew Wrather overthink the orthogonal cultural spectacles of the 2026 Super Bowl, considering the halftime show headlined by Bad Bunny and the perennial cultural battlefield of the commercials, this year weaving narratives of authenticity amidst a sea of grifting. The halftime show is approached first through its stagecraft and camera work which create an transporting, immersive environment for TV audiences (though probably not for stadium-goes). The presence of Gaga is debated.], [And what does it all tell us about America in 2026? Maybe it's that "in a world where nostalgia is commodified, even a minion can become a cultural icon."], [Download (MP3)], [List of Super Bowl Halftime Shows (Wikipedia)], [Episode 22: DIY Naked News], [style="margin: 5px 0; padding: 10px; background: #eee;">], [style="margin: 0; padding: 0;"> Episode 919: The Who is the What; the When is the Why originally appeared on Overthinking It , the site subjecting the popular culture to a level of scrutiny it probably doesn't deserve. [Latest Posts | Podcast (iTunes Link)]],), insert-map: (:), word-count: 185, edited-for-length: false, debug-mode: false,)

standard-article(title: [Episode 122: Hercules], author: [Nate DiMeo], source-name: [The Memory Palace], images: (), paragraphs: ([Order The Memory Palace book now, dear listener. On Bookshop.org , on Amazon.com , on Barnes & Noble , or directly from Random House . Or order the audiobook at places like Libro.fm .], [The Memory Palace is a proud member of Radiotopia from PRX. Radiotopia is a collective of independently owned and operated podcasts that's a part of PRX, a not-for-profit public media company. If you'd like to directly support this show, you can make a donation at Radiotopia.fm/donate.], [This episode originally dropped in 2018.], [We start with Facing the Obstacles , from Robert Simonson's score to The Final Member.], [1979 by Deru. The Julianna Barwick remix of This Will Destroy You's The Puritan.],), insert-map: (:), word-count: 136, edited-for-length: false, debug-mode: false,)

standard-article(title: [Island city-builder Nova Roma is out now, and I'd have drowned all my Romans already if it weren't for those pesky gods], author: [Edwin Evans-Thirlwell], source-name: [Rock Paper Shotgun], images: (), paragraphs: ([I have two dreams as mayor of an island town in Nova Roma , the new early access city-building game from Lion Shield and Hooded Horse. One is to erect a fantastic water network for my people - a sturdy yet poetic lattice of aqueducts, following their deft gradations down from the mountain rivers to cisterns gracefully spaced amid the insulae, forums, circuses and temples. In my reborn Rome, no populous bathhouse, tinkling fountain, or humble latrine shall ever run dry. With my other hand, I shall raise mighty dams, diverting the rivers away from my walls to avoid flooding in times of heavy rainfall, while exposing velvety expanses of buildable, tillable soil.], [My citizens will learn to treat water frivolously, swilling and pissing it away in their decadence, much as they did in the Rome of old. The fools! For when my empire of hydration is complete, I will ascend the slopes and whimsically commission one final dam. Trusting in my stewardship – for what reason have I given them to disobey? - the citizens shall toil day and night to finish the structure. Then, when the last stone is laid and the sluices slam shut, they shall gaze in horror as a tidal wave engulfs their fair metropolis and sweeps all their precious bloody bathhouses away.], [Read more],), insert-map: (:), word-count: 220, edited-for-length: false, debug-mode: false,)

standard-article(title: [20 fabulous family spring days out in the UK], author: [Fiona Kerr], source-name: [The Guardian Travel], images: (), paragraphs: ([Join the Famous Five in Dorset, relive Springwatch in the Peak District ... our selection of Easter treats will keep all the family entertained], [Spring has arrived at Wicken Fen , one of Europe's most important wetlands, and with it the first summer migrants. Chiffchaffs are usually the earliest, with their rhythmic song ringing out across the fens. Then, if the weather is mild, blackcaps and willow warblers might join them. Listen closely, especially early morning or at dusk, for the foghorn-like calls of the booming bittern across the reedbeds. There's a pushchair- and wheelchair-friendly boardwalk around Sedge Fen, and wheelchair-accessible wildlife hides. Look out for the electric blue flash of a kingfisher, and male marsh harriers performing their dramatic sky-dancing flights as the breeding season gets under way, before the cuckoos arrive in late April. From £1 0 adults , £5 children (under-5s free), nationaltrust.org.uk], [Continue reading...],), insert-map: (:), word-count: 149, edited-for-length: false, debug-mode: false,)

My citizens will learn to treat water frivolously, swilling and pissing it away in their decadence, much as they did in the Rome of old.

Edwin Evans-Thirlwell

standard-article(title: [I got tired], author: [Scott Hanselman], source-name: [Scott Hanselman], images: (), paragraphs: ([I have been blogging here for the last 20 years. Every Tuesday and Thursday, quite consistently, for two decades. But last year, without planning it, I got tired and stopped. Not sure why. It didn't correspond with any life events. Nothing interesting or notable happened. I just stopped.], [I did find joy on TikTok and amassed a small group of like-minded followers there. I enjoy my YouTube as well, and my weekly podcast is going strong with nearly 900 (!) episodes of interviews with cool people. I've also recently started posting on Mastodon (a fediverse (federated universe)) Twitter alternative that uses the ActivityPub web standard . I see that Mark Downie has been looking at ActivityPub as well for DasBlog (the blog engine that powers this blog) so I need to spend sometime with Mark soon.], [Being consistent is a hard thing, and I think I did a good job. I gave many talks over many years about Personal Productivity but I always mentioned doing what "feeds your spirit." For a minute here the blog took a backseat, and that's OK. I filled that (spare) time with family time, personal projects, writing more code, 3d printing, games, taekwondo, and a ton of other things.], [Going forward I will continue to write and share across a number of platforms, but it will continue to start here as it's super important to Own Your Words . Keep taking snapshots and backups of your keystrokes as you never know when your chosen platform might change or go away entirely.], [I'm still here. I hope you are too! I will see you soon.], [Related Links:], [Do they deserve the gift of your keystrokes?], [Do you have a digital or social media will?], [], [style="clear: both; padding-top: 0.2em;">],), insert-map: (:), word-count: 299, edited-for-length: false, debug-mode: false,)

standard-article(title: [Episode 125: Snakes!], author: [The Memory Palace], source-name: [The Memory Palace], images: (), paragraphs: ([The Memory Palace is a proud member of Radiotopia.], [Herman's Malt from Jeff Russo's score to Fargo Season 2.], [David Goes Hunting from Larry Groupe's score to Straw Dogs.], [A snippet of Idea of Order at Kyson Point from an album Brian Eno did with Tom Rogerson.], [A bit of Il Tamoure dei Bambini from Piero Umiliani's score to Le Isole Dell'amore], [A Naga Swaram-Snake Charmers Melody from Folk Music of India], [Finishes out on Completely Gone from Ludwig Goransson's score to Everything Everything.], [Learn about your ad choices: dovetail.prx.org/ad-choices],), insert-map: (:), word-count: 122, edited-for-length: false, debug-mode: false,)

standard-article(title: [Episode 220: The Zipper], author: [Nate DiMeo], source-name: [The Memory Palace], images: (), paragraphs: ([The Memory Palace is a proud member of Radiotopia from PRX. Radiotopia is a collective of independently owned and operated podcasts that's a part of PRX, a not-for-profit public media company. If you'd like to directly support this show and independent media, you can make a donation at Radiotopia.fm/donate. I have recently launched a newsletter. You can subscribe to it at thememorypalacepodcast.substack.com .], [Swiming by Explosions in the Sky Walking Song by Kevin Volans and the Netherlands Wind Ensemble I Walk on Guilded Splinters by Johnny Jenkins Seduction by the Balanescu Quartet Lunette by Les Baxter and Dr. Samuel J. Hoffman Running Around by Buddy Ross September by Giles Lamb], [Notes This episode was pieced together from a ton of little fragments but I wanted to steer folks to a couple of resources in particular: this excellent article from a few years back in the Toronto Star by Katie Daubs, and this documentary from filmmaker, Amy Nicholson, that primarily uses the Zipper as a way to talk about changes at Coney Island but has some great details from Harold Chance and his sons.],), insert-map: (:), word-count: 184, edited-for-length: false, debug-mode: false,)

standard-article(title: [Two Small Sculptures (The Met Residency Episode 8)], author: [The Memory Palace], source-name: [The Memory Palace], images: (), paragraphs: ([Nate DiMeo was the Metropolitan Museum of Art's Artist in Residence for 2016/2017. He produced 8 episodes inspired by the collection and by the museum itself. This is the eigth episode of that residency.], [This residency is made possible by the Metropolitan Museum of Art's Chester Dale Fund.], [This episode is written and produced and stuff by Nate DiMeo with engineering assistance from Elizabeth Aubert. Its Executive Producer is Limor Tomer, General Manager Live Arts, The Metropolitan Museum of Art.], [Hiawatha , Edmonia Lewis, 1868.], [Minnehaha , Edmonia Lewis, 1868.], [Learn about your ad choices: dovetail.prx.org/ad-choices],), insert-map: (:), word-count: 108, edited-for-length: false, debug-mode: false,)

{

Episode 126 (The 8th Story)

The Memory Palace

The Memory Palace is a proud member of Radiotopia.

Then a smidge of N. Y. C. from the original Broadway cast recording of Annie.

The Stars vs. Creatures by Colleen.

Wiese by Roedelius & Arnold Kasar.

Then there's some chaos built out of Missing Piece from the score to Broken City, Fog Tropes by Ingram Marshall, Spindrift by Colin Stetson, and Longing for a Frozen Sky by Ernst Reijseger.

Learn about your ad choices: dovetail.prx.org/ad-choices

}

{

ANALYSIS

IN BRIEF

Joy Gendusa, Entrepreneur: Email volume is up 33.9% — here's how we increased CTR 10% and turned cold leads into 631 replies and 95 calls per week.

Nate DiMeo, The Memory Palace: The Memory Palace is a proud member of Radiotopia from PRX.

Music

Dave Pajo/Aerial M does Plastic Energy Man

Patricia Rossborough played To a Wild Rose

Mal Waldron plays Warm Canto

We hear Muff Gets a Share from Joel P. West's score to Band of Robbers

We hear another song I absolutely love, Turned Out I Was Everyone, by Sasami

We finish on Popcorn and Life from Ben Sollee's lovely score to Maidentrip.

Reece Bithrey, Rock Paper Shotgun: The Asus ROG Strix Scope II 96 Wireless has been a long-time favourite mechanical keyboard of mine. That's because of its highly functional, near-full-size layout, its smooth, lubricated switches, and just the sense that the entire package feels thoughtful and well-put-together. I tried looking for a deal for in in the UK, sadly to no avail, though our US pals will be happy to learn it's currently $130 from Amazon US in their Spring Sale (that, for some strange reason, is happening over a week after the UK one ended).

Read more

Oisin Kuhnke, Rock Paper Shotgun: It sounds like Kingdom Come: Deliverance 2 developer Warhorse Studios' future projects won't be translated entirely by human hands. Earlier today, a Reddit post was shared to the game's subreddit from Max Hejtmánek, a Czech to English translator and editor on the developer's most recent game, where he claimed that yesterday, March 27th, he was laid off "in favour of using AI for all translations going forward."

Read more

Nate DiMeo, The Memory Palace: The Memory Palace is a proud member of Radiotopia from PRX.

Music

Inception by radio.string.quintet.vienna

Julie With by Group Listening

Nice Breeze Isn't It? by friend of the show, Simon Rackham

Wet by Taylor Deupree

Times Like This II by Jean Kopperud and Stephen Gosling

Broad Channel by Bing and Ruth

Cradle (with Akira) by ghost and tape

Lithosphere by Caoimhin O Raghellagh

and by Caoimhin O Raghelagh and Thomas Bartlett

Notes

You can find the website I mentioned here ; it's a one-stop shop, really, for information on the 6888t. .

The Memory Palace, The Memory Palace: If you enjoy this story, please tell a friend about The Memory Palace.

Thank you kindly.

Learn about your ad choices: dovetail.prx.org/ad-choices

Glenn Garner, Deadline Hollywood: Mary Beth Hurt, the actress known for roles in The Age of Innocence and Six Degrees of Separation, has died. She was 79. The 3x Tony-nominated actress' daughter Molly Schrader, whom she shared with husband Paul Schrader, announced that Hurt died on Saturday after she was diagnosed with Alzheimer's in 2015. "Yesterday morning we lost [...]

The Memory Palace, The Memory Palace: If you enjoy this story, do tell someone about The Memory Palace.

Thanks.

Nate

Learn about your ad choices: dovetail.prx.org/ad-choices

Martin Fowler, Martin Fowler: class="img-link">

Erik Doernenburg is the maintainer of CCMenu: a Mac application that shows the status of CI/CD builds in the Mac menu bar. He assesses how using a coding agent affects internal code quality by adding a feature using the agent, and seeing what happens to the code.

}

Deep Standard

Vol. 1, No. 016 · 2026-03-30

Curated and composed automatically.